

PORTADA

Técnicas de Programación

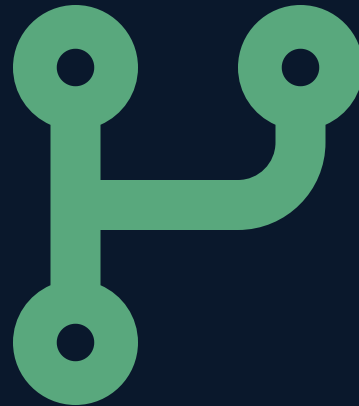
Unidad 3 — Técnicas de Programación: GitHub Colaborativo y Mejores Prácticas

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Dos personas editan el mismo archivo al mismo tiempo, se lo pasan por USB, y una hora después nadie sabe cuál versión es la buena.



TRANSICIÓN

De escribir código limpio a compartirlo sin perderlo

Ya sabes escribir clases bien diseñadas (SOLID) y legibles (código limpio). Falta la parte que hace posible trabajar **en equipo** sobre ese mismo código: un sistema de control de versiones. **Git** es la herramienta que registra el historial de cambios; **GitHub** es la plataforma donde ese historial se guarda en un repositorio remoto y se coordina con otras personas.

CONCEPTO

Git no es GitHub

Son dos cosas distintas que se usan juntas:

	Qué es
Git	Sistema de control de versiones que corre en tu computador. Registra el historial de cambios de un proyecto (quién cambió qué, cuándo y por qué).
GitHub	Plataforma en la nube que aloja repositorios Git de forma remota, y agrega herramientas de colaboración: Pull Requests, revisión de código, issues.

Existen alternativas a GitHub (GitLab, Bitbucket) — todas funcionan sobre el mismo Git por debajo.

CONCEPTO

El ciclo básico: clone, add, commit, push, pull

Un repositorio remoto en GitHub se sincroniza con tu copia local mediante estos comandos:

```
git clone https://github.com/usuario/proyecto.git # copiar el repo remoto
git add archivo.java                             # marcar cambios para guardar
git commit -m "Agrega validación de entrada"      # guardar los cambios localmente
git push                                           # subir los commits al remoto
git pull                                           # traer los cambios de otros
```

`commit` guarda en tu historial local; `push` / `pull` sincronizan ese historial con GitHub — sin `push`, tus commits nunca salen de tu computador.

CONTRAEJEMPLO

Todo el equipo commiteando directo a `main`

Sin ramas, cualquier cambio a medio terminar de una persona queda visible de inmediato para todo el equipo — incluyendo código que no compila o que rompe una funcionalidad que otros ya están usando. Un conflicto de dos personas editando la misma línea se vuelve mucho más difícil de resolver cuando ya está mezclado en `main` que cuando estaba aislado en una rama propia.



CONCEPTO

Flujo con ramas: crear, trabajar, integrar

1. Crear una rama a partir de `main` para la funcionalidad que vas a trabajar.
2. Hacer commits en esa rama a medida que avanzas.
3. Subir la rama a GitHub con `git push origin nombre-rama`.
4. Abrir un **Pull Request** pidiendo integrar esa rama a `main`.
5. Un compañero revisa el código (*code review*) y lo aprueba o pide cambios.
6. Una vez aprobado, se hace *merge* a `main`.

CONCEPTO

Mensajes de commit que sí sirven

El historial de commits es documentación: cada mensaje debe explicar **qué cambió y por qué**, en modo imperativo y en una línea corta (idealmente menos de 50 caracteres), con detalle adicional en el cuerpo si hace falta.

Mal	Bien
"cambios"	"Corrige cálculo de IVA en Factura"
"fix"	"Valida que el correo no esté vacío"
"asdf"	"Agrega pruebas para GestorPedidos"

CONCEPTO

Pull Request y code review

Un **Pull Request** (PR) es una solicitud formal de integrar una rama a otra (normalmente a `main`), con un espacio para que otras personas **revisen el código antes de integrarlo**. En el code review se comenta: claridad del código, posibles bugs, si sigue las convenciones del equipo (nombres, SOLID, formato) — no es una crítica personal, es una revisión técnica constructiva sobre el código, nunca sobre quien lo escribió.

CONCEPTO

Conflictos de merge

Un **conflicto** ocurre cuando dos ramas modificaron las mismas líneas de un archivo de formas distintas, y Git no puede decidir automáticamente cuál versión conservar. Git marca el conflicto directamente en el archivo:

```
<<<<<< HEAD
double iva = precio * 0.19;
=====
double iva = precio * IVA_COLOMBIA;
>>>>>> feature/constante-iva
```

Resolver el conflicto significa editar el archivo a mano, decidir qué versión (o combinación) queda, borrar las marcas

<<< / == / >>> , y hacer un nuevo commit.

CONCEPTO

Higiene del repositorio: **.gitignore** y **README**

Dos archivos que todo repositorio colaborativo debería tener:

- **.gitignore** : lista de archivos que Git debe ignorar (archivos compilados `.class` , configuración local del IDE, credenciales) — evita que basura o información sensible termine en el historial compartido.
- **README.md**: la puerta de entrada al proyecto — qué hace, cómo instalarlo/ejecutarlo, y cómo contribuir. Es lo primero que ve cualquier persona nueva que abre el repositorio.

CASO

GitLab.com, 2017: cuando falla hasta el respaldo

En enero de 2017, un ingeniero de GitLab.com intentó eliminar un directorio en un servidor secundario para resolver un problema de replicación, pero ejecutó el comando de borrado en el **servidor de producción** por error, eliminando cerca de 300 GB de datos de usuarios en vivo. Al intentar restaurar desde respaldo, el equipo descubrió que **los 5 mecanismos de backup configurados habían fallado silenciosamente** — ninguno estaba funcionando correctamente.

GitLab publicó el incidente en vivo (streaming) y un post-mortem público detallado — un ejemplo, dentro del propio ecosistema de control de versiones, de por qué los procesos y controles alrededor del repositorio importan tanto como el código mismo.

TRANSICIÓN

De la teoría al flujo real

Ninguno de estos comandos vale mucho si no se practican en equipo. Ahora es el turno de simular, en parejas, el flujo completo: rama, commit y Pull Request.

INTERACCIÓN

Simulen el flujo de colaboración

En parejas, con un repositorio de prueba en GitHub:

- Persona A crea una rama `feature/saludo` , agrega un método simple, hace commit con un mensaje claro y la sube (`push`).
- Persona A abre un Pull Request hacia `main` .
- Persona B revisa el PR en GitHub, deja al menos un comentario, y lo aprueba.

10 minutos · en sala de Zoom asignada

SÍNTESIS

Lo que hay que llevarse de hoy

1. Git registra el historial de cambios; GitHub aloja ese historial en un repositorio remoto y agrega herramientas de colaboración.
2. Trabajar en ramas separadas evita pisarse el trabajo entre compañeros — nunca `commit`ear directo a `main` en equipo.
3. El Pull Request y el code review son el punto donde el equipo revisa el código antes de integrarlo.
4. Un conflicto de merge se resuelve editando el archivo a mano y haciendo un nuevo commit.
5. `.gitignore` y README no son opcionales: son higiene básica de cualquier repositorio colaborativo.



CIERRE

Antes de la próxima sesión

- Practicar el flujo completo (rama → commit → push → PR) en un repositorio propio.
- Revisar el README de algún proyecto público en GitHub y evaluar si cumple lo visto hoy.

Próxima sesión: Documentación y optimización de código — cierre de la Unidad 3.

